Module-VII
Lecture-II
Functional and Structural Testing

1. **Introduction**

In the last lecture we learnt that Digital VLSI testing is to verify if logic functionality is as per the specifications. For example, for the 2 input NAND gate the test would comprise only the 4 input combinations and verifying the output. The time required to verify anything more than logic functionally is beyond practical limits in terms of ATE time, man hours, cost of the chip etc. Also, the quality of test solution (i.e., proper binning of normal and faulty chips) is acceptable for almost all classes of circuits. This is called *Functional Testing.*

Now let us consider a digital circuit with 25 inputs which does bit wise ANDing of the inputs. The black box of the circuit is shown in Figure 1. The complete functional test is given in Table 1.

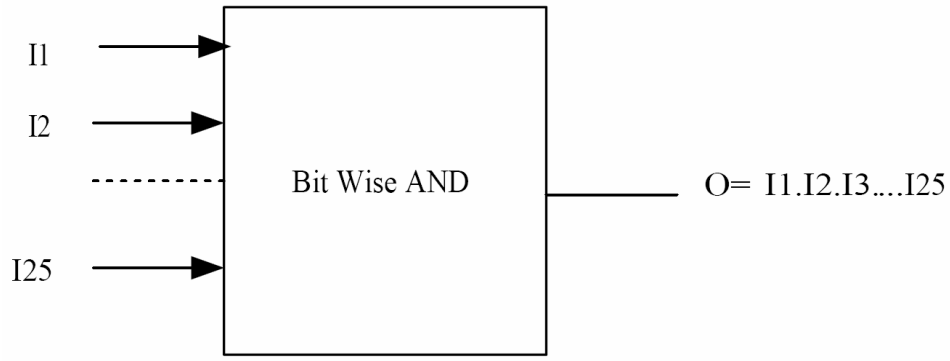


Figure 1. Circuit for Bit wise ANDing

Table 1. Test patterns for functional testing of circuit in Figure 1

Test Pattern No.	Test Pattern	Output
1	00000000000000000000000	0
2	00000000000000000000001	0
……	……..	0
2^{25}	11111111111111111111111	1

We need to apply 2^{25} test patterns to complete the test. If we apply 1000000 patterns per second (Mega Hz Tester), then time required is 33 Seconds per chip. In a typical scenario about 1 million chips are to be tested in a run, thereby taking about 33000000 Seconds or 550000 Hours or 22916 Days or 62 years. So one can understand the complexity when a circuit has 100+ inputs. So for a typical circuit even *Functional Testing* cannot be performed due to extremely high testing time.

To solve this issue we perform *Structural Testing,* which takes many fold less time compared Functional Testing yet maintaining the quality of test solution. Structural testing, introduced by Eldred, verifies the correctness of the specific structure of the circuit in terms of gates and interconnects. In other words, structural testing does not check the functionality of the entire circuit rather verifies if all the structural units (gates) are fault free. So structural testing is a kind of functional testing at unit (gate) level.

In the next section we elaborate the gain in test time for structural testing using the example of the 25 input ANDing circuit. Also, the cost that needs to be paid is illustrated using the same example.

2. **Structural Testing: An Example Of 25 Input Bit Wise ANDing Circuit**

To perform structural testing a white box view of the circuit (i.e., gate level implementation) is required. Following that functional testing would be performed on the individual gates. Figure 2 shows a gate level implementation of the 25 input bit wise ANDing circuit (of Figure 1).

Structural testing for the circuit would comprise the patters discussed below:

1. Testing of the 5 input AND gate G1 with the input patterns as given in Table 2.

Table 2. Test patterns for testing of gate G1

	Test Pattern (I1,I2,I3,I4,I5)	Output (OG1)
1	00000	0
2	00001	0
……	……..	0
32	11111	1

2. Repeat similar test patterns for all the 5 input AND Gates G2, G3, G4, G5 and G6.

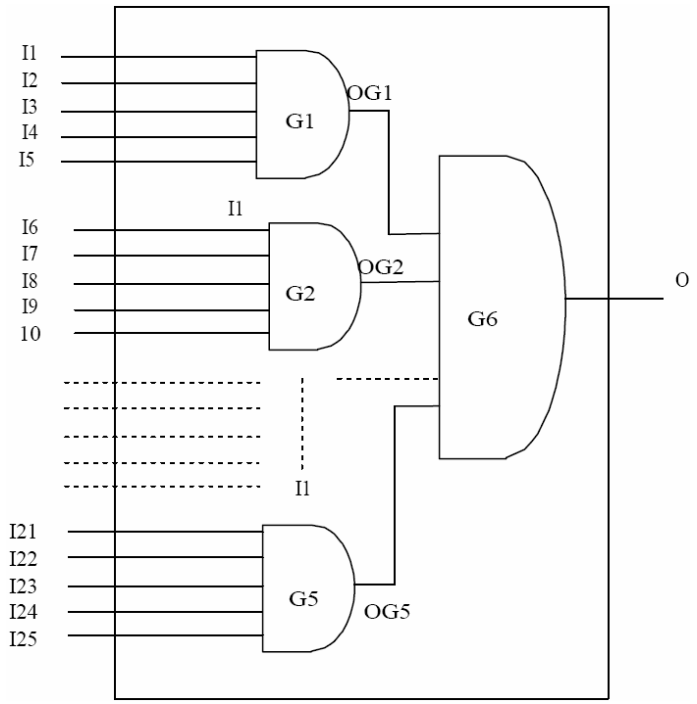


Figure 2. Gate level implementation for Bit wise ANDing circuit

So number of test patterns required are 6.2^5 (=160), which is many fold smaller than those required for functional testing (2^{25}). In this case, time required for testing the circuit the using a 1 Mega Hz Tester is 0.000016 seconds and for a million samples is 16 seconds. Now we can easily see the benefits for structural testing over functional testing.
However, some piece is to be paid for the benefits obtained, as listed below

1. Each individual gate is tested; however, the integration is not tested. From history of several thousand cases of chips being fabricated and tested, it was observed that the quality of test solution given by structural testing is acceptable.
2. To test the individual gates, controlling and observing values of intermediary nets in a circuit becomes mandatory, which adds to extra pins and hardware. For example, to test Gate G1 (for circuit shown in Figure 2) we need to apply signals at pins I1 through I5 which are primary inputs and brought out of the chip as pins. But we also need to observe the output at net OG1, which is difficult as it is not a primary output and is internal to the chip. So, for structural testing, involving testing of Gate G1 individually, OG1 is to be brought out of the chip as a special output (test) pin. Similarly for individual testing of the gates G2 through G5, lines OG2 through OG5 are to be made observable by additional pin outs. *So for structural testing some internal nets are to be made observable by extra pin outs.*
3. Now for testing G6, the problem is different. The output of the gate G6 can be observed as it is a primary output. However, to test G6, inputs are to be given through internal nets OG1 through OG5. It may be noted that these nets are outputs of other AND gates and to drive these nets directly to a required value, not only they are to be brought out of the chip as pins but also are to be decoupled from the corresponding AND gates. *So for structural testing some internal nets are to be made controllable by extra pin outs and circuitry.* Controllability is achieved by adding extra 2-1 Multiplexers on these nets. This is illustrated in Figure 3 (as bold lines boxes). During normal operation of the circuit the Test Mode signal (i.e., connected to select lines of the 2-1 Multiplexers) is made 1; the outputs of the AND gates (G1-G5) are passed to the inputs of the AND gate G6. When G6 is to be tested, Test Mode signal is made 0; inputs to Gate G6 are

decoupled from Gates G1 through G5 and now they can be driven directly by additional input pins TI1 through TI5. These additional circuits (Multiplexers and pins) to help in structural testing are called Design for Testability (DFT).

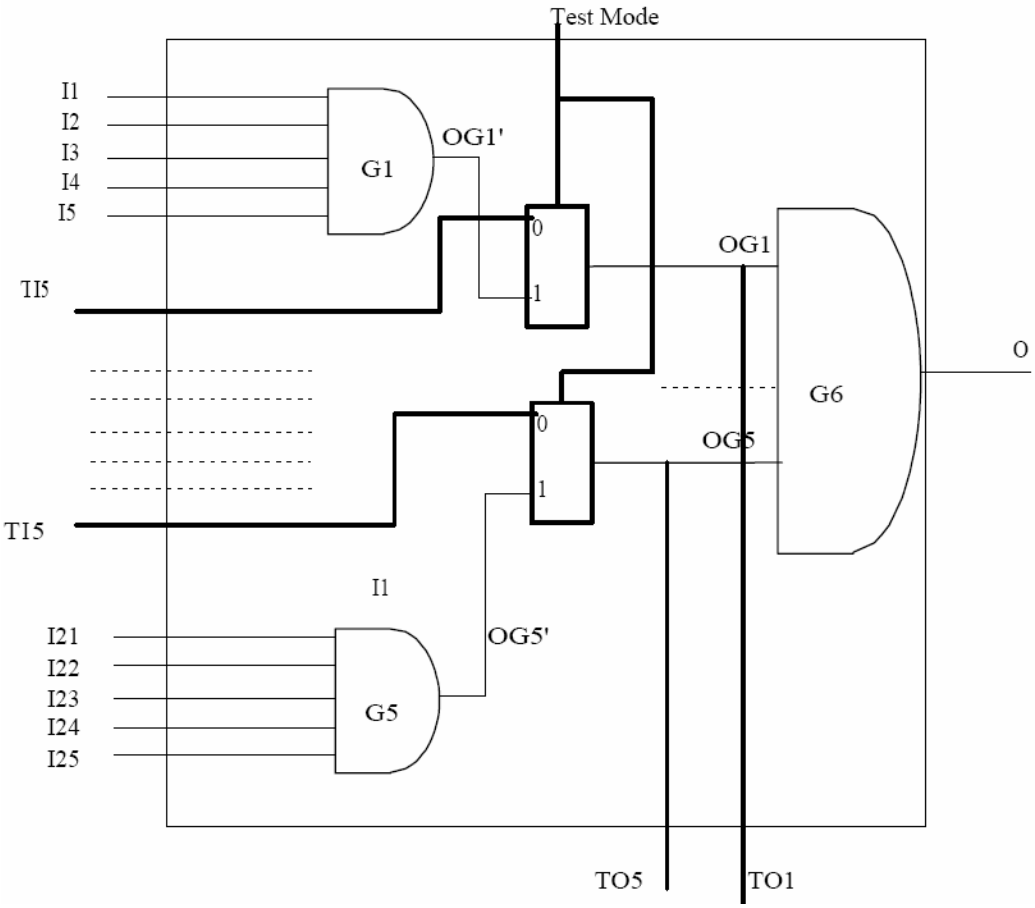


Figure 3. Extra pins and hardware for structural testing of circuit of Figure 2.

It is to be noted that a circuit with about a million internal lines needs a million 2-1 Multiplexers and same number of extra pins. This requirement is infeasible. So let us see in steps how these problems can be minimized, yet maintaining quality of test solution. In the next section we illustrate using an example, to show internal memory can reduce extra pin outs for DFT.

3. **Structural Testing With Internal Memory: An Example Of A 32 Bit Adder**

A 32-bit adder shown is Figure 4 requires 2^{32} test patterns for exhaustive functional testing. As discussed in last section, structural testing of the 32-bit adder can reduce the testing time to a great extent. In this case, we consider a full adder as one structural unit (like the AND gates for the circuit in Figure 2). The implementation of the 32-bit adder in terms full adders (i.e., structural units) is also shown in Figure 4. Now we illustrate how structural testing of the 32-bit adder can be done with only $8(=2^3)$ test patterns and 3 extra pin outs. The 32-bit adder with DFT circuitry is shown in Figure 5.

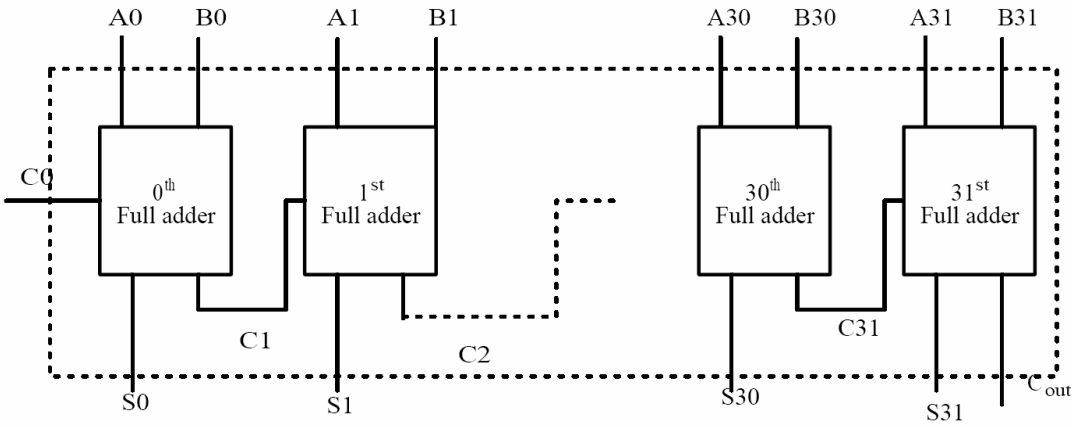


Figure 4. A 32-bit adder

The DFT circuitry comprises, 2 31-bit shift registers, 31 2-1multiplexers and 3 pin outs. One shift register (called input register) provides inputs to the ``carry input'' bits of the individual adders during test and the other shift register (called output register) latches outputs from the ``carry output'' bits of the individual adders. In the modified 32-bit adder, the carry input to the ith (full) adder is multiplexed with the ith -bit of the input shift register, $1 \le i \le 31$. During normal operation of the 32-bit adder, the multiplexers connect the carry input of the ith (full) adder to the carry output of the $(i-1)^{th}$ (full) adder, $1 \le i \le 31$. However, during test, the multiplexers connect the carry input of the ith (full) adder to the output of the ith -bit of the input shift register, $1 \le i \le 31$. The values in the shift register are fed externally. It may be noted that by this DFT arrangement all the (full) adders can be controlled individually as direct access is provided to the carry inputs of the adders; inputs other than carry are already controllable. Hence, testing in this case would be for each (full) adder individually and that requires 8 test vectors as each of the 32 full adders can be tested in parallel.

Correct operations of each of the full adders are determined by looking at the sum and the carry outputs. Sum outputs are already available externally and hence no DFT circuit is required to make them directly observable. For the carry outputs, however, another similar DFT arrangement is required to make them observable externally. This would require the output (31 bit parallel load and) shift register where the carry output bit of the $(i\text{-}1)^{th}$ adder is connected to the ith input of the output shift register, $1 \le i \le 31$. Once the values of all the carry bits are latched in the register, which is done in parallel during test, they are shifted out sequentially. In this case a full adder is tested functionally and structural information is used at the cascade level.

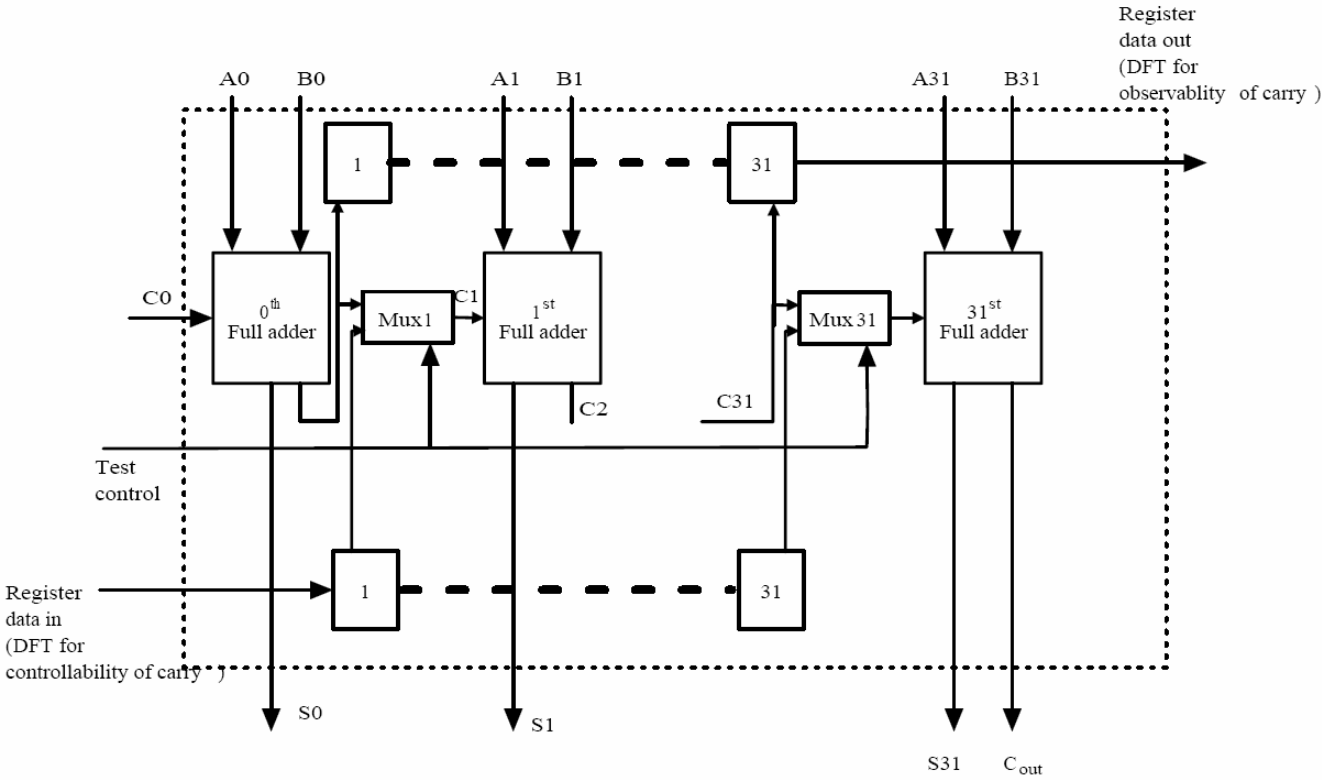


Figure 5. A 32-bit adder with DFT circuitry

Now let us see the gains and costs paid by the DFT circuitry (shift registers) for structural testing

- Gain:
    - Instead of 2^{32} test patters for functional testing, only 2^3 patterns are enough for structural testing
    - The number of extra pins for structural testing is only 3. It many be noted that if shift register is not used, then extra pins required are 64 as carry output bit of the full adders are to be brought out for observation and carry in bit of the full adders are to be brought out for sending the required test signal (controllability).
- Price
    - 2-1 Multiplexers and registers are required for each internal net to controlled
    - Registers required for each internal net to observed
    - 3 extra pin outs

From Section 2 and Section 3 we note that by the use of internal registers, the problem of huge number of extra pins could be solved but it added to requirement of huge size of shift registers (equal to number of internal nets). In a typical circuit there are tens of thousand of internal lines, making the on-chip register size and number of 2-1 multiplexers extremely high. So addition to cost of a chip by such a DFT is unacceptable.
So our next target for achieving an efficient structural testing is the one with less number of on-chip components and yet maintaining the quality of test solution. Structural testing with Fault Models is the answer to the requirement. In the next section we will study Fault Models and then see why DFT requirement is low when structural testing is done with fault models.

Before going to next section, to summarize, ``*structural testing is functional testing at a level lower than the basic input-output functionality of the system*". For the example of the bitwise ANDing circuit, unit for structural testing was gates and for the case of 32-bit adder it was full adders. In general, in the case of digital circuits, structural testing is ``*functional testing at the level of gates and flip-flops*". Henceforth in this course, basic unit of a circuit for structural testing would be logic gates and flip-flops.

4. **Structural Testing with Fault Models**

Structural testing with fault models involves verifying each unit (gate and flip flop) is free from faults of the fault model.

4.1 What is Fault Models

A model is an abstract representation of a system. The abstraction is such that modeling reduces the complexity in representation but captures all the properties of the original system required for the application in question.

So, fault model is an abstraction of the real defects in the silicon such that

- the faults of the model are easy to represent
- should ensure that if one verifies that no faults of the model are in the circuit, quality of test solution is maintained.

In perspective of the present discussion, the following definitions are important

Defect: A defect in a circuit is the unintended difference between the implemented hardware in silicon and its intended design.

Error: An error is an "effect" of some "defect."

Fault: Abstraction of a "defect" at the fault modeling level is called a fault.

Let us consider an AND G1 of Figure 2, where due to incomplete doping of metal, net I1 is left unconnected with the gate; this is illustrated in Figure 5. This unconnected net I1 is the defect. Error is, when I1=1, I2=1,I3=1,I4=1, I5=1 but OG1=0 (should be 1 in normal case). Fault is, net I1 is stuck at 0 (when gate is modeled at binary logic level).

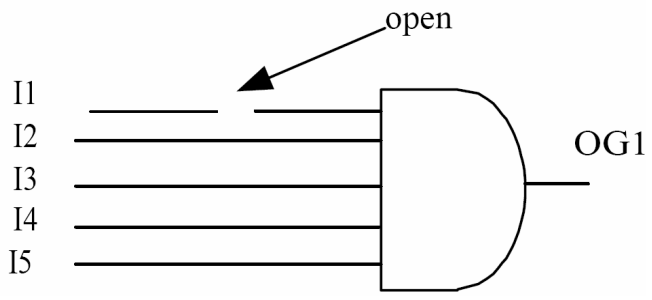


Figure 5: AND gate with one net open

4.2 Widely Accepted Fault Models

Many fault models were proposed [1,2], but the ones widely accepted are as follows

- **Stuck-at fault model:** In this model, faults are fixed (0 or 1) value to a net which is an input or an output of a logic gate or a flip-flop in the circuit. If the net is stuck to 0, it is called stuck-at-0 (s-a-0) fault and if the net is stuck to 1, it is called stuck-at-1 (s-a-1) fault. If it is assumed that only one net of the circuit can have a fault at a time, it is called single stuck-at fault model. Without this assumption, it is called multiple stuck-at fault model. This has been observed though fabrication and testing history, that if a chip is verified to be free of single stuck-at faults, then it can be stated with more than 99.9% accuracy that there is no defect in the silicon or the chip is functionally (logical) normal. Further, single-stuck at fault is extremely simple to handle in terms of DFT required, test time etc.; this will be detailed in consequent lectures. So single stuck-at fault model is the most widely accepted model.
- **Delay fault model:** Faults under this model increase the input to output delay of one logic gate, at a time.
- **Bridging Fault:** A bridging fault represents a short between a group of nets. In the mot widely accepted bridging fault models, short is assumed between two nets in the circuit. The logic value of the shorted net may be modeled as 1-dominant (OR bridge), 0-dominant (AND) bridge. It has been observed that if a circuit is verified to be free of s-a faults, then with high probability it can be stated that there is no bridging fault also.

Next we elaborate more on the stuck-at fault model which is most widely accepted because of its simplicity and quality of test solution.

4.3 **Single Stuck-at Fault Model**

For single stuck-at fault model it is assumed that a circuit is an interconnection (called a netlist) of Boolean gates. A stuck-at fault is assumed to affect only the interconnecting nets between gates. Each net can have three states: normal, stuck-at-1 and stuck-at-0. When a net has stuck-at-1 (stuck-at-0) fault it will always have logic 1(0) irrespective of the correct logic output of the gate driving it. A circuit with *n* nets can have *2n* possible stuck-at faults, as under single-at fault model it is assumed that only one location can have a stuck-at-0 or stuck-at-1 fault at a time. The locations of stuck-at faults in the AND gate G1 (in the circuit of Figure 2) are shown in Figure 6 with black circles. G1 is a 5 input AND gate thereby having 5 input nets and an output net. So, there are 12 stuck-at faults possible in the gate.

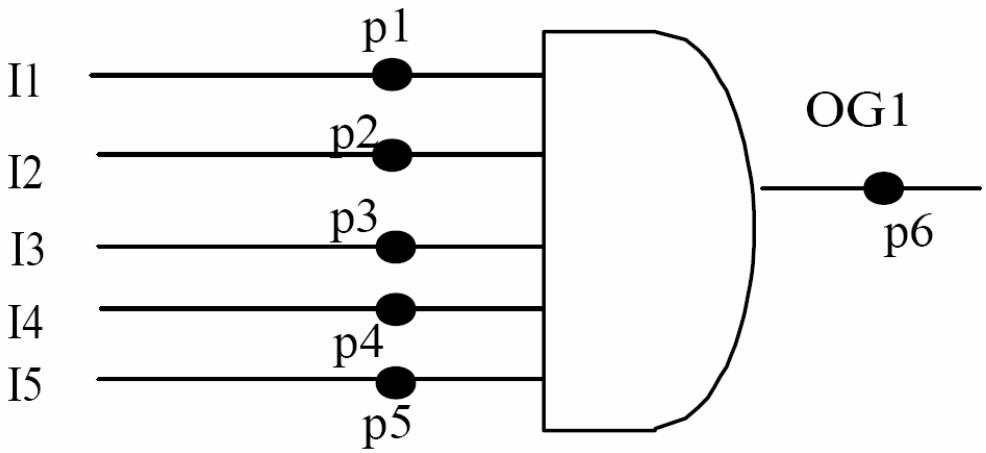


Figure 6: AND gate with stuck-at fault locations

Now let us consider another circuit shown in Figure 7 with fanouts, and see the stuck-at fault locations especially in the fanout net.

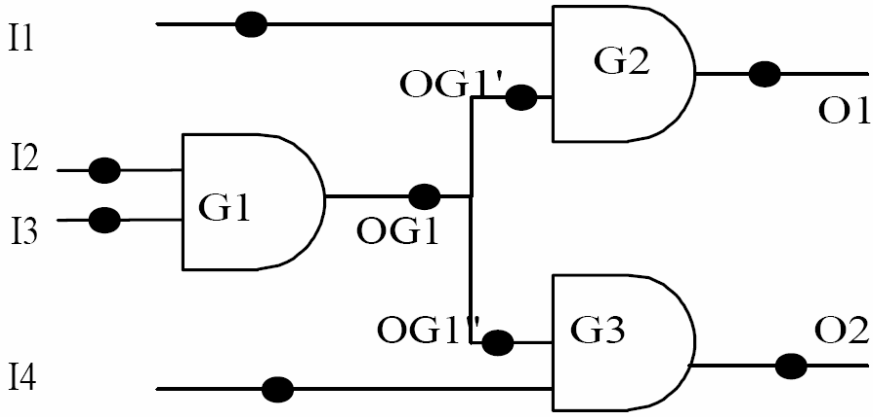


Figure 7: Stuck-at fault locations in a circuit with fanouts

Output of gate G1 drives inputs of gate G2 and G3. Here, the output net of G1 has three fault locations for stuck-at faults rather than one, as marked in Figure 7 by black circles. One point corresponds to output of gate G1 (OG1) and the other two to the inputs of gate G2 (OG1') and G3 (OG1"). In other words, a net having fanout to *k* gates will have *k+1* stuck at fault locations; one for the gate output (stem of the fanout) and the others for the inputs of the gates that are driven (branches of the fanout). When the stuck-at fault is at the output of the gate (OG1, in this case) then all the nets in the fanout have the value corresponding to the fault. However, if fault is in one fanout (OG1' or OG1", in this case) then only the corresponding gate being driven by the branch of the fanout is effected by the stuck-at fault and others branches are not effected.

Now the question remains, "if fanout along with all its branches is a single electrical net, then why fault in a branch does not affect the others". The answer is again by history of testing of chips with single-stuck at fault model. It may be noted that a net getting physically stuck may not occur. When we perform structural testing with single stuck-at fault model, we verify that none of the sites have any stuck-at fault. Assuring this ensures with 99.9% accuracy that the circuit has no defect. Considering different branches of a fanout independent, more locations for stuck-at faults are created. Testing history on stuck-at faults has shown that this increased number of locations is required to ensure the 99.9% accuracy.

To summarie, single stuck-at fault model is characterized by three assumptions:

1. Only one net is faulty at a time.

2. The faulty net is permanently set to either 0 or 1.

3. The branches of a fanout net are independent with respect to locations and affect of a stuck-at fault.

In general, several stuck-at faults can be simultaneously present in the circuit. A circuit with *n* lines can have 3^n-1 possible stuck line combinations; each net can be: s-a-1, s-a-0, or fault-free. All combinations except one having all nets as normal are counted as faults. So handling multiple stuck-at faults in a typical circuit with some hundreds of thousands of nets is infeasible. As single stuck-at fault model is manageable in number and also provides acceptable quality of test solution, it is the most accepted fault model.

Now we illustrate structural testing with single stuck-at fault model.

4.4 **Structural Testing with Stuck-at Fault Model**

We will illustrate structural testing of stuck-at faults of the circuit in Figure 2.

First let us consider s-a-0 in net I1, as shown in Figure 8. As the net I1 is stuck-at-0, we need to drive it to 1 to verify the presence/absence of the fault. To test if a net is stuck-at-0(stuck-at-1), obviously it is to be driven to opposite logic value of 1 (0). Now all other inputs (I2 through I5) of G1 are made 1, which propagates the effect of fault to OG1; if fault is present then OG1 is 0, else 1.To propagate the effect of fault to O, in a similar way, all inputs of G2 though G5 are made 1. Now, if fault is present, then O is 0, else 1. So, I1=1, I2=1,….,I25=1, is a test pattern for the stuck-at-0 fault at net I1.

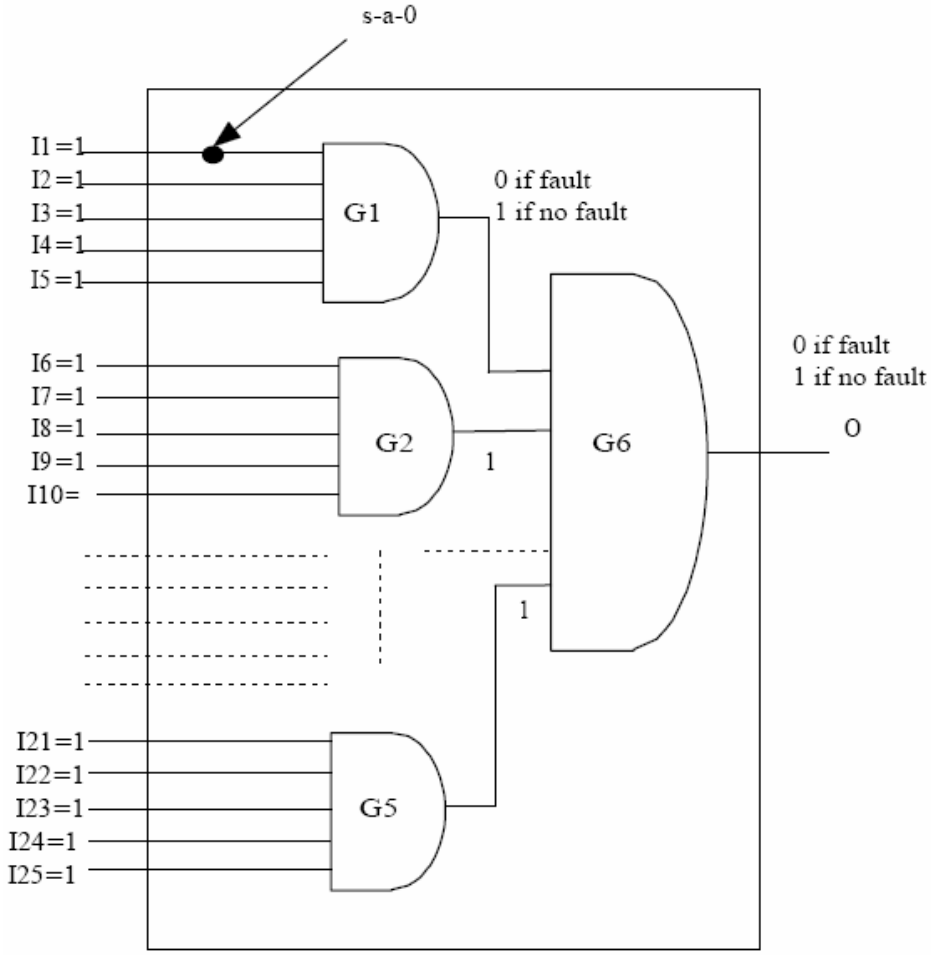


Figure 8. s-a-0 fault in net I1 with input test pattern

Now let us consider s-a-1 in net I1, as shown in Figure 9. As the net I1 is stuck-at-1, we need to drive it to 0 to verify the presence/absence of the fault. Now all other inputs (I2 through I5) of G1 are made 1, which propagates the effect of fault to OG1; if fault is present then OG1 is 1, else 0. To propagate the effect of fault to O, in a similar way, all inputs of G2 though G5 are made 1. Now, if fault is present, O is 1, else 0. So, I1=0, I2=1,….,I25=1, is a test pattern for the stuck-at-1 fault at net I1.

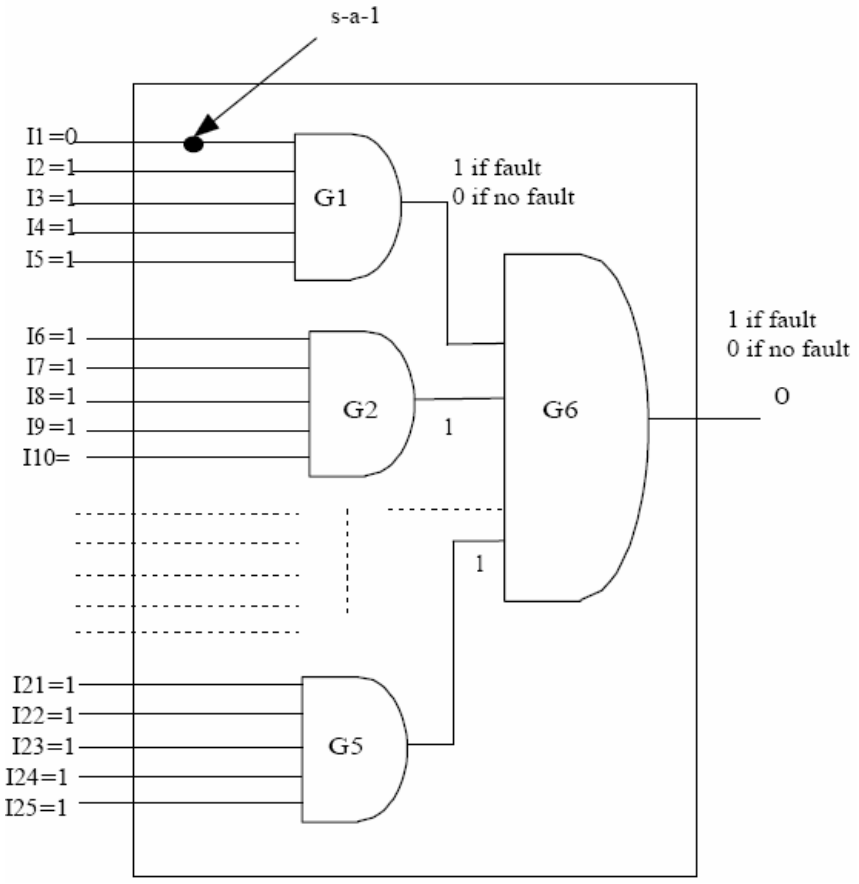


Figure 9. s-a-1 fault in net I1 with input test pattern

Now let us consider s-a-1 in an internal net (output of G1), as shown in Figure 10. As the net output of G1 is stuck-at-1, we need to drive it to 0 to verify the presence/absence of the fault. So at least on input of G1 is to be made 0; at G1, I1=0 and I2 through I5 are 1. To propagate the effect of fault to O, all inputs of G2 though G5 are made 1. Now, if fault is present, then O is 1, else 0. So, I1=0, I2=1,….,I25=1, is a test pattern for the stuck-at-1 fault at net output of G1. It is interesting to note that same test pattern I1=0, I2=1,….,I25=1 tests both s-a-1 at net I1 and output of G1. In other words, in structural testing with stuck-at fault model, one test pattern can test more than one fault.

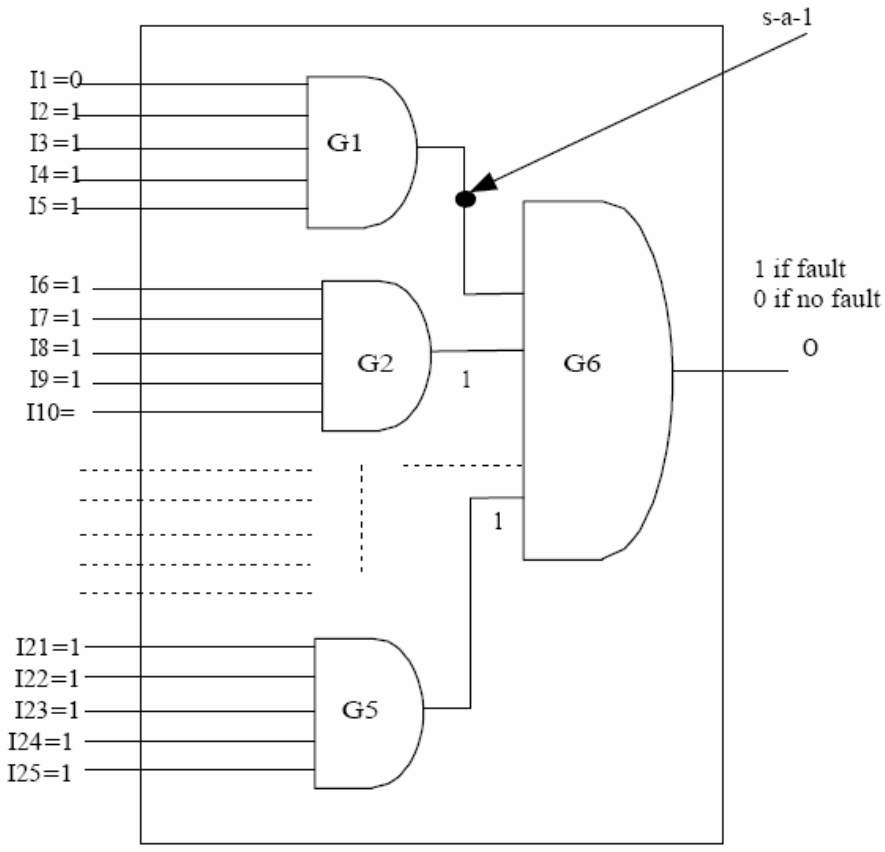


Figure 10. s-a-1 fault in net OG1 with input test pattern

Now let us enumerate the gains and price paid for structural testing with stuck-at fault model

- Gains
    - No extra pin outs or DFT circuitry like 2-1 Multiplexers and shift resisters for controlling and observing internal nets
    - Low test time as one test pattern can test multiple stuck-at faults
- Price
    - Functionality is not tested, even for the units (gates and Flip-flops). However, testing history reveals that even with this price paid, quality of test solution is maintained.

To conclude, Table 2 compares Structural and Functional Testing

Table 2. Comparison of structural and functional testing

Functional testing	**Structural Testing**
Without fault models.	With fault models.
Manually generated design verification test patterns.	Automatic test pattern generation (ATPG).
Slow and labor intensive.	Efficient and automated.
Fault coverage not known	Fault Coverage is a quantified metric.
More Test Patterns	Less Test Patterns
Can be applied at the operating speed.	Difficult to be applied at the speed the design is expected to work.

.

References

[1] J. A. Abraham and H.C. Shih. Transistor-level test generation for physical failures in CMOS circuits. Design Automation Conference, pages 243–249, 1986.

[2] M. Abramovici, M.A. Breuer, and A.D. Friedman. Digital Systems Testing and Testable Design. Wiley-IEEE Press, 1994.

Questions and Answers

1. What are the problems of structural testing without fault models, if the units are as atomic as gates and as large as arithmetic block like 32-bit adder and 64-multiplier

Answer:

If structural testing is done without fault models and the units are small (like gates), then there is lot of internal nets (interconnecting the gates) to be controlled or observed. One 2-1 Multiplexer and 1 external pin is required for all internal nets (which are inputs to gates) which are to be controlled and one 1 external pin is required for all internal nets (which are outputs of gates) which are to be observed.

On the other hand if the units are large blocks like 32-bit adder and 64-multiplier, then the numbers of internal nets are only the interconnections between these blocks. The number of nets required to interconnect large blocks are many times smaller than those required to interconnect gates. So multiplexers and extra pins are less compared to the case when gate is treated as an uni.

Structural testing is functional testing at unit level (if done without fault models). Functional testing requires exponential number of test patterns with respect to the number of inputs of the units. In case of gates, inputs are generally four or less, however, in case of large blocks inputs may be as large as 128 (32 for the adder and 64 for the multiplier, in this example). So test patters for structural testing with large blocks as units is many fold more in number than the case when the unit is a gate.

2. How many silicon level defects result in nets getting stuck? Explain relevance of stuck-fault model from that perspective

Answer

Very rarely, silicon level defects result in nets getting stuck. However, from history of testing of fabricated chips, it is observed that if one verifies the absence of all stuck-at

faults in the nets of a circuit, then with more than 99.9% probability it may be stated that there is no defect.